IS1201: Programming & Problem Solving

2. C Programming

Viraj Welgama

Programming Language

- A **programming language** is a way for people to give instructions to a computer using words and symbols the computer can understand.
- It's like a language we use to **tell the computer what to do**, such as doing calculations, showing messages, or saving data.

Generation of Programming Languages

- First Generation (1GL) – Machine Language
  - Binary code (0s and 1s)
  - Directly understood by the computer
  - Very fast but hard to read/write
- Second Generation (2GL) – Assembly Language
  - Uses mnemonics (e.g., MOV, ADD)
  - Easier than machine code
  - Still hardware-specific
- Third Generation (3GL) – High-Level Languages
  - More human-readable (e.g., C, Java, Python)
  - Portable across machines
  - Requires compilers or interpreters

Generation of Programming Languages

- Fourth Generation (4GL)
  - Even closer to natural language
  - Focus on what to do, not how
  - Examples: SQL, MATLAB

- Fifth Generation (5GL)
  - Based on solving problems using constraints and logic
  - Used in AI and expert systems
  - Examples: Prolog, LISP

Program Paradigm

- Programming paradigm refers to how a program is written in order to solve a problem.
- Different programming paradigms are;
  1. Imperative paradigm (Fortan, Pascal, C)
     - developer writes code that **explicitly describes how a program operates**, step by step, to achieve a desired outcome.
  2. Functional paradigm (scala)
     - computation is treated as the **evaluation of mathematical functions** and avoids changing state or mutable data.
  3. Logic paradigm (prolog)
     - style of programming where programs are written as a set of **logical statements and rules**, and computation is performed by asking questions (queries) about those statements.
  4. Object-Oriented paradigm (java, C++, Python)

How the Machine Understand your Code?

- Even though you write instructions in English-like languages, the machine still understands only **binary signals** (0s and 1s).

- So, someone needs to **translate your code** into a form the machine can understand.

- There are two types of translators:
  1. Compiler – uses *compilation*
  2. Interpreter – uses *interpretation*

Compilation

- **Compilation** is the process of **converting code** written in a programming language (like C or Java) into **machine language** (binary) that a computer can understand.
  - You write code in a high-level language (e.g., C)
  - A compiler reads your code
  - It translates the code into machine code (executable file)
  - The computer runs the machine code

- Happens **before** the program runs
- Makes the program run faster

Interpretation

- **Interpretation** is the process of **running a program line by line** using an **interpreter**, without converting the whole program into machine code first.
  - You write code in a high-level language (e.g., Python)
  - An interpreter reads and executes the code line by line
  - The computer performs the actions as it reads the code

- Happens **while** the program runs
- Easier to test and debug
- Slower than compiled code
- Used in languages like **Python**, **JavaScript**, **Ruby**

We will learn C!

- Why?

Why C?

1. Middle level language
   - You will understand how a computer works.
2. Compiled language
   - fast
3. Machine independent
   - portable
4. Basic concepts of programming
   - You will find it much easier to learn other programming languages.
5. Nothing can beats C by performance
   - OS like Windows, Linux are still written in C
6. Opportunity to work on open source projects that impact millions of people.

About…

- C is a very powerful general-purpose programming language.
- It is fast, portable and available in all platforms.
- C language used for wide range of applications from Operating systems like Windows, Unix and iOS to software that is used for creating 3D movies.
- C programming is highly efficient.
  - That's the main reason why it's very popular despite being more than 40 years old.
- If you are new to programming, C is a good choice to start your programming journey.

History!

- BCPL:
  - **Stands for:** Basic Combined Programming Language
  - **Purpose:** to write compilers for the other programming languages
  - **By:** Mr. Martin Richards
  - **At:** AT & T Bell labs (founded by Alexander Graham Bell, USA)
  - **Year:** 1966
  - **New Features:** introducing {} to group set of statements instead of BEGIN, END
  - **Limitations:** follow too low level coding styles, not suitable for developing large software applications.
  - **Influenced to:** B programming language

History!

- B programming language:
  - **Stands for:** 1st character of the BCPL
  - **Purpose:** to reincarnate BCPL language and write system programs
  - **By:** Mr. Ken Thompson
  - **At:** AT & T Bell labs
  - **Year:** 1969
  - **Limitations:** not quite suitable to implement UNIX OS, not suitable for developing moderately complex applications.
  - **Influenced to:** C programming language

History!

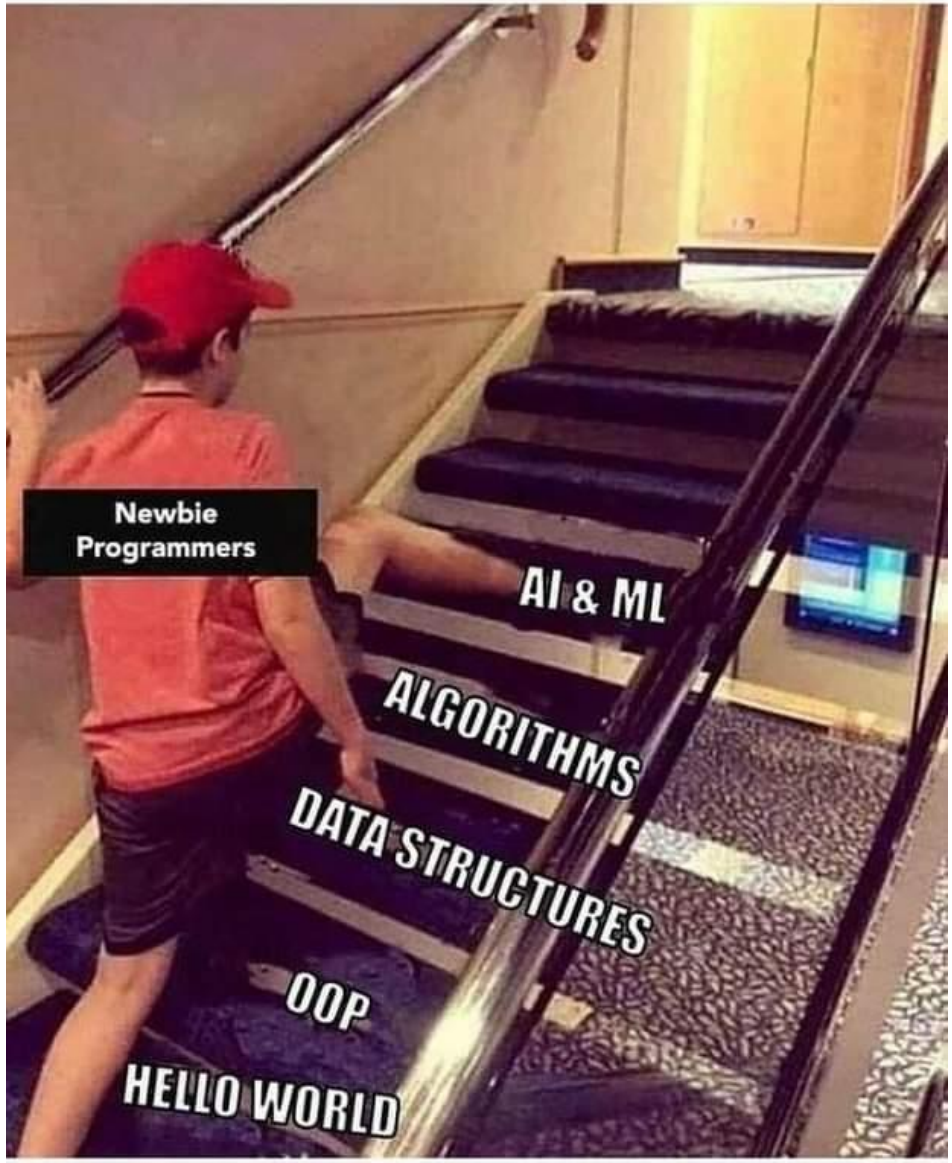
- C programming language:
  - **Stands for:** 2nd character of the BCPL
  - **Purpose:** re-implement the UNIX operating system
    - UNIX OS was written on assembly level language
  - **By:** Mr. Dennis Ritchie, Mr. Stephen C. Johnson
  - **At:** AT & T Bell labs
  - **Year:** 1972
  - **New Features:** B with data types.
  - **Limitations:** not suitable to represent real world entities and solve most of the real world problems
  - **Influenced to:** CPP (C with Classes, hence C++)

History!

- **1983:**
  - ANSI (American National Standards Institute) established standard specification for C.
- **1989:**
  - Based on the standard specification, IEEE release new C version with the name ANSI C, standard C or C89.
- **1990:**
  - ISO (International Organization for Standardization) adopted ANSI C standard and named C90.
- **Dec 2011:**
  - Stable and standard of C language has been released with the name C11.

Acknowledgment

- https://www.programiz.com/c-programming


Newbie Programmers
AI & ML
ALGORITHMS
DATA STRUCTURES
OOP
HELLO WORLD

When you start coding in a new language without reading the documentation

Your 1st C Program…

```c
#include <stdio.h>

int main() {
    printf("Hello C!\n");
    return 0;
}

```

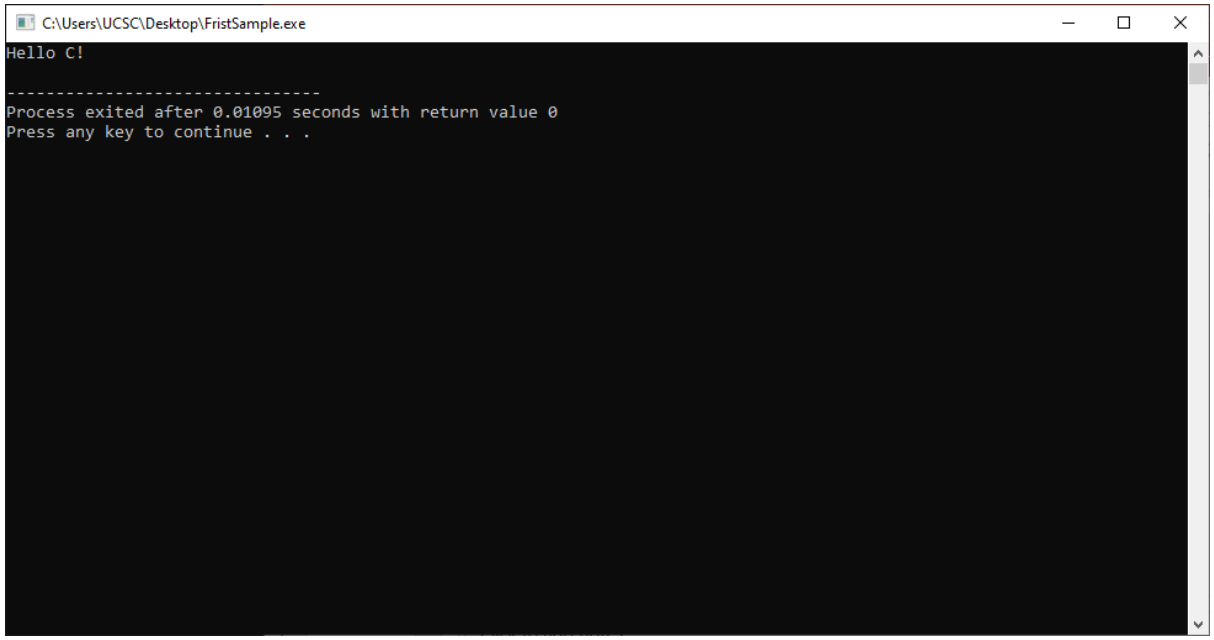
- To compile your program…
  - > gcc hello.c
- This will creates an output file a.exe (in Windows) which is directly executable.
- To execute the program…
  - >a.exe

Your 1st C Program…

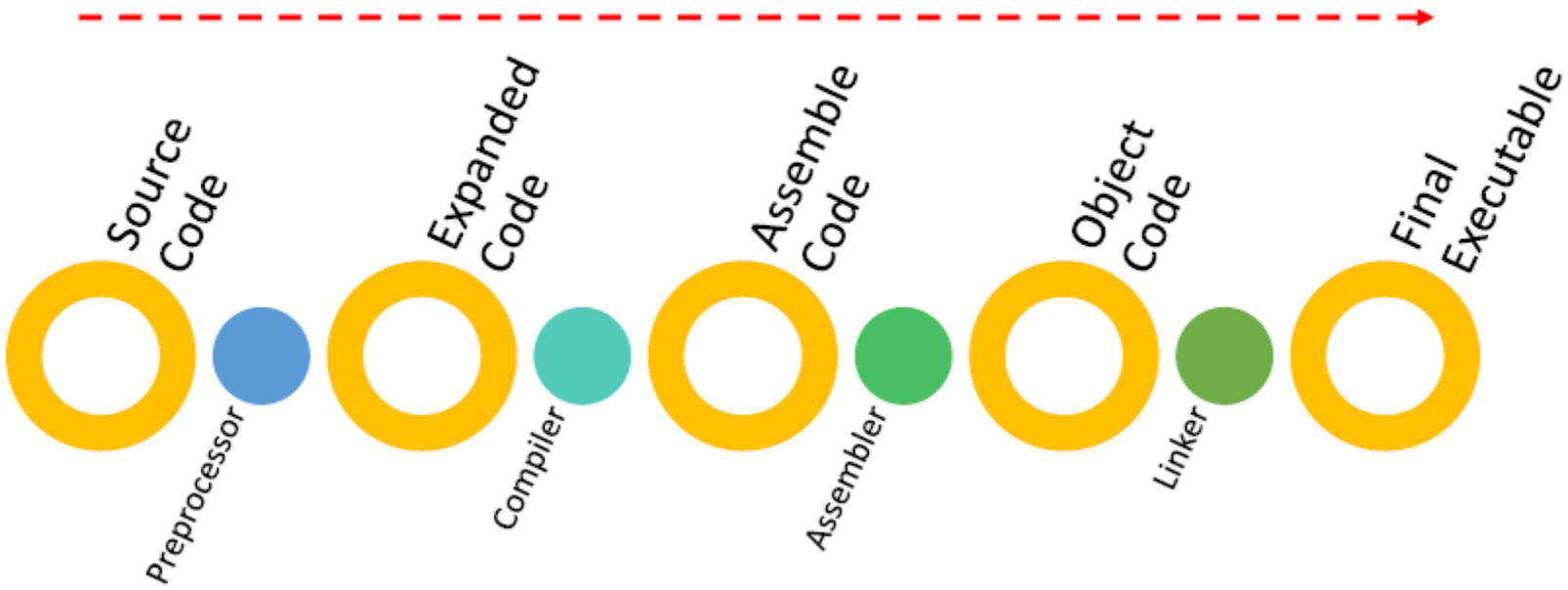
```c
#include <stdio.h>

int main() {
    printf("Hello C!\n");
    return 0;
}

```



Compiling Steps…



Preprocessor

- #include represent a preprocessor directive
- Any line in a C program that starts with a # at the beginning of a source file is a preprocessor directive.
- These directive are processed by the preprocessing step.
- So, the line 1 of our program includes the contents of the standard input and output header file (stdio.h) to our source program.
- To see the preprocessor output
  - >gcc –E hello.c > filename
- The option –E when used, stops the complilation process after the preprocessing stage.

Intermediate Steps…

- Too see the output after the compilation before the assembler is run
  - >gcc –S hello.c
- The above step will result in an assembly named hello.s

- To see the output after the execution of the assemble and before the linker
  - >gcc –c hello.c
- The above step will result in an object file named hello.o
- To get the final executable after the linking process
  - >gcc hello.o
- The above will create an executable file named a.exe

Your 1st C Program…

```c
#include <stdio.h>

int main() {
    printf("Hello C!\n");
    return 0;
}

```

- In this program, we have used printf() function which displays the text inside the quotation mark. Since printf() is defined in stdio.h, you need to include stdio.h.
- In C programming, the code execution begins from the start of main() function (doesn't matter if main() isn't located at the beginning).
- The code inside the curly braces { } is the body of main() function. The main() function is mandatory in every C program.

Your 1st C Program…

```c
#include <stdio.h>

int main() {
    printf("Hello C!\n");
    return 0;
}

```

- The printf() is a library function that sends formatted output to the screen (displays the string inside the quotation mark).
- Notice the semicolon at the end of the statement.
- The return statement return 0; inside the main() function ends the program. This statement isn't mandatory. However, it's considered good programming practice to use it.

Key Notes…

- All C program starts from the main() function and it's mandatory.
- You can use the required header file that's necessary in the program.
- C is case-sensitive; the use of uppercase letter and lowercase letter have different meanings.
- The C program ends when the program encounters the return statement inside the main() function.
- The statement in a C program ends with a semicolon (;).

Built in Libraries…

- C programming has several in-built library functions to perform input and output tasks.
- Two commonly used functions for I/O (Input/Output) are printf() and scanf().
- The scanf() function reads formatted input from standard input (keyboard) whereas the printf() function sends formatted output to the standard output (screen).

C Output

```c
#include <stdio.h> // this is needed to run printf()

int main() {
    printf("C Programming"); //displays the content inside quotation
    return 0;
}
```

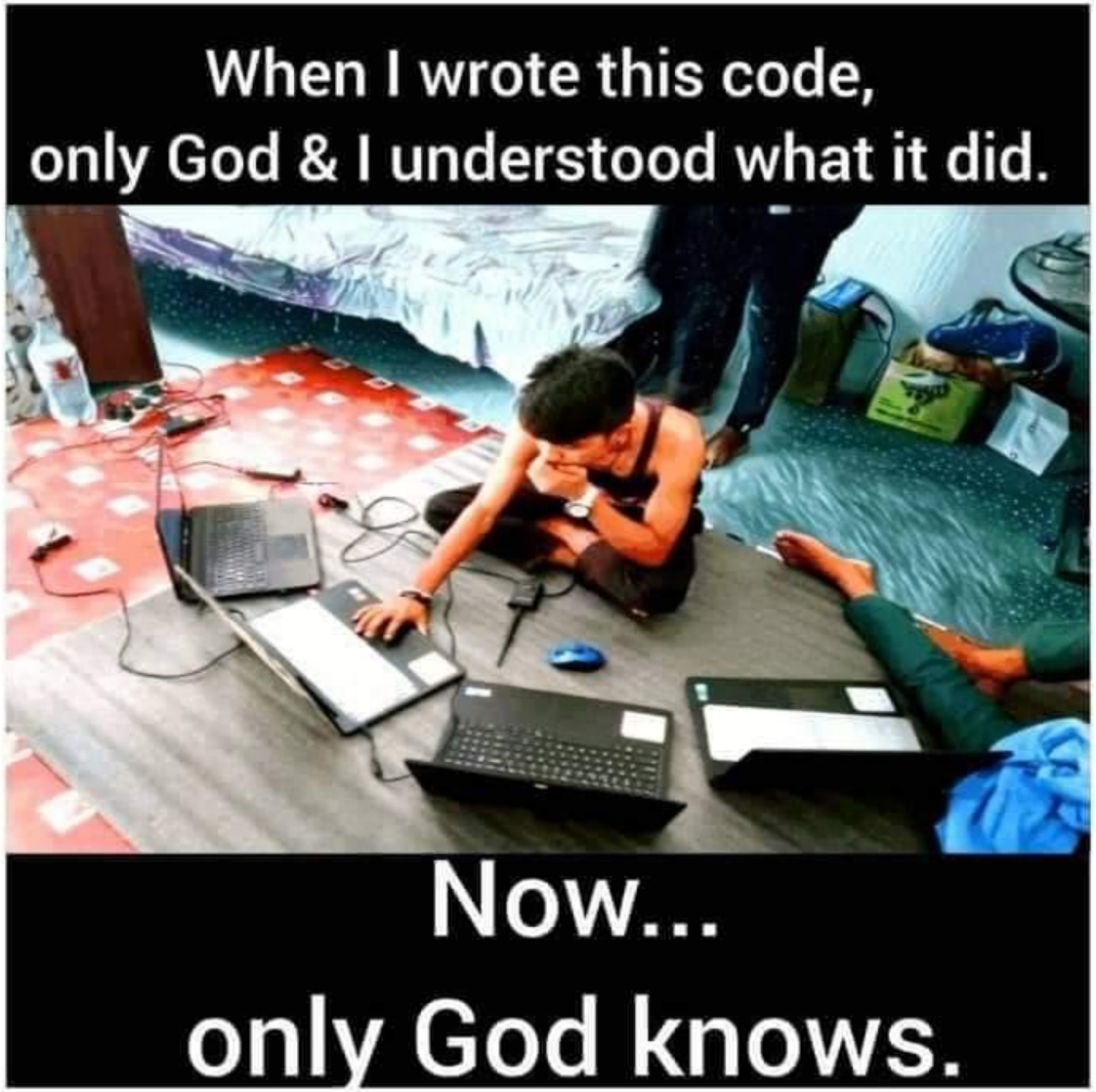
```c
#include <stdio.h>

int main() {
    int testInt = 10;
    printf("Number = %d", testInt);
    return 0;
}
```

- Inside the quotation of printf() function, there is a format string "%d" (for integer). If the format string matches the argument (testInt in this case), it is displayed on the screen.

How this program works?

- All valid C program must contain the main() function. The code execution begins from the start of main() function.
- The printf() is a library function to send formatted output to the screen.
- The printf() function is declared in "stdio.h" header file.
- Here, stdio.h is a header file (standard input output header file) and #include is a preprocessor directive to paste the code from the header file when necessary.
- When the compiler encounters printf() function and doesn't find stdio.h header file, compiler shows error.
- The return 0; statement is the "Exit status" of the program. In simple terms, program ends.


When I wrote this code,
only God & I understood what it did.
Now...
only God knows.

Comments

- As programs get bigger and more complicated, they get more difficult to read. Formal languages are dense, and it is often difficult to look at a piece of code and figure out what it is doing, or why.
- For this reason, it is a good idea to add notes to your programs to explain in natural language what the program is doing.

- These notes are called comments , and they start with The // symbol or /* */ for C.

Problems

1. Write a C program to add two Integers
2. Write a C program to read and print your name